Student Registrations System at XYZ University

Introduction

In today's digital age, most universities have adopted sophisticated database systems to manage their students' information. University XYZ is no exception and has developed its own Student Registrations System to keep track of student records. However, the system lacks a crucial feature that can make student registration easier - **a register and login page**. Currently, students need to visit the university's enrolment department in person to register their information manually. This approach is not only time-consuming but can also lead to errors in record-keeping.

To address this issue, the university has decided to undertake a project to develop a database system that will allow students to register and manage their information quickly. The project aims to create a user-friendly system for students to register their details, such as personal information, academic records, and extracurricular activities. The system will also provide features to allow students to view and update their records, track their academic progress, and communicate with the university staff.

The project is an essential step for the university toward digital transformation, making it easier for students to manage their academic life. The new system will improve the student experience and help the university staff streamline administrative processes, reduce errors, and enhance data security. With the successful implementation of the new system, the university can expect to see increased student satisfaction, improved administrative efficiency, and a better overall academic environment.

The project will involve creating a Conceptual E-R model and a Relational model for the database, using Structured Query Language in MySQL database, creating a simple GUI for only a register and login page of an application using a programming language to connect to the MySQL database, and performing select, insert, update, and delete statements through GUI.

Entities

It would help if you considered defining **at least six entities**. For example, the following six entities **may be** required for the database system:

Student - This entity represents a student and includes attributes such as student ID, name, address, phone number, email address, and date of birth.

Example:

Student ID	First Name	Last Name	Address	Phone Number	Email	Date of Birth
1	John	Doe	123 Main St.	(555) 555-1234	johndoe@email.com	01/01/2000

Course - This entity represents a course and includes attributes such as course ID, course name, instructor name, start time, end time, and room number.

Example:

Course ID	Course Name	Instructor Name	Start Time	End Time	Room Number
1	Introduction to Computer Science	Dr. Smith	9:00am	10:30am	Room 101

Enrolment - This entity represents a student's enrolment in a course and includes attributes such as enrolment ID, student ID, course ID, and grade.
Example:

Enrolment ID	Student ID	Course ID	Grade
1	1	1	A

Schedule - This entity represents a student's schedule and includes attributes such as student ID, course ID, start time, end time, and room number.
Example:

Student ID	Course ID	Start Time	End Time	Room Number
1	1	9:00am	10:30am	Room 101

Department - This entity represents a department and includes attributes such as department ID and name.
Example:

Department ID	Department Name
1	Computer Science

Professor - This entity represents a professor and includes attributes such as professor ID, name, address, phone number, and email address.
Example:

Professor ID	First Name	Last Name	Address	Phone Number	Email
1	Alex	Doe	369 Sun St.	(888) 333-6699	alexdoe@email.com

Task 1: Create a Conceptual E-R Model for the Database
The primary objective of the first task of this project is to develop a conceptual E-R model for the database that will define the structure of the database, relationships, and attributes. To create the E-R model, at least six entities, such as students, faculty, courses, departments, grades, and extracurricular activities, need to be defined along with their relationships and attributes.
These entities will be the foundation of the database and ensure that the data can be accurately captured and easily accessed. In addition, the E-R model will allow University XYZ to understand the data flow within the system, ensuring that the database structure meets the organization's needs.
A tool like Lucidchart can be used to create the E-R model. The E-R model created will serve as a blueprint for designing the database, guiding the development of data storage and access capabilities for University XYZ.
In conclusion, creating a conceptual E-R model is a critical first step in designing a database system that is functional and efficient for University XYZ. Defining at least six entities, relationships, and attributes will help ensure that the system meets the organization's needs, allowing for accurate data capture and easy access.
Note: several other tools are available for creating E-R models besides Lucidchart. Some of the most popular tools for creating E-R diagrams include:
Draw.io: A free and open-source diagramming tool that can create a wide range of diagrams, including E-R diagrams.
Gliffy: A cloud-based diagramming tool that allows users to create E-R and other diagrams.

SmartDraw: A diagramming tool that provides a range of templates, including E-R diagrams, to help users create professional-looking diagrams quickly.
Creately: A cloud-based diagramming tool that offers a range of templates, including E-R diagrams and collaboration features for teams.
ConceptDraw DIAGRAM: A diagramming tool that provides a range of templates and libraries, including E-R diagrams, and allows users to create custom templates.
These tools offer functionality similar to Lucidchart, each with strengths and weaknesses. The choice of tool depends on factors such as the user's preferences, budget, and the project's specific needs.

Task 2: Create a Relational Model for the Database in MySQL
In this task, you will create a relational model for the database in MySQL. This approach is an essential step as the relational model will define the structure of the database, including the tables, columns, and relationships between them. The relational model will be based on the conceptual E-R model created in Task 1, which identifies at least six entities that will be used in the database.
To create the relational model, the project team will need to define the tables used in the database, along with their respective columns and data types. In addition, the relationships between the tables must also be defined, such as the primary key-foreign key relationships that link records in different tables.
For example, the project team can create a "students" table using SQL script. The "students" table may contain columns such as student_id, first_name, last_name, address, phone_number, email, and date_of_birth. The student_id column will serve as the primary key for the table. Other columns, such as first_name, last_name, and email, will be defined as varchar data types, while columns, such as date_of_birth, will be defined as the date data type.
It would help if you created additional tables as the conceptual E-R model required, ensuring that each table is appropriately linked to other tables through primary key and foreign key relationships. By creating a well-defined relational model, the database can be structured easily to query and manage, allowing for efficient data storage and retrieval. Following are two pieces of SQL script to create two tables as two examples for you:

```
CREATE TABLE students (
  student_id INT PRIMARY KEY,
  first_name VARCHAR(50),
  last_name VARCHAR(50),
  address VARCHAR(100),
  phone_number VARCHAR(20),
  email VARCHAR(50),
  date_of_birth DATE
);

CREATE TABLE courses (
  course_id INT PRIMARY KEY,
  course_name VARCHAR(50),
  instructor_name VARCHAR(50),
  start_time TIME,
  end_time TIME,
  room_number VARCHAR(20)
);
```

Task 3: Use Structured Query Language
Task 3 requires using Structured Query Language (SQL) to manage the data in the database. SQL is a standard language used for managing relational databases. You will use SQL to create at least six tables and insert, update, delete, and retrieve data from the database.

In this task, you will focus on performing operations related to the **registrations and login of a student**. Therefore, you must write SQL statements such as select, insert, update, and delete for the Student table. In addition, you should write at least six SQL queries to retrieve information from the Student table based on specific criteria. For instance, you might want to retrieve the names of all students in the table or retrieve a student's password based on their ID.
In addition to retrieving data, you will write one SQL statement to insert data into the Student table, one SQL query to update data in the Student table based on the student's ID, and one SQL query to delete a student based on their ID.
By writing SQL queries, you will show how to retrieve, manipulate, and manage data in the database. This approach will help you better understand how the database system works and how data can be organized, queried, and managed efficiently.

Task 4: Create a Simple GUI for an Application
This task is focused on creating a simple graphical user interface (GUI) for an application that can interact with the MySQL database created in Task 3. The GUI will allow users to select, insert, update, and delete statements on the database. In addition, the students are required to implement the **register and login page of the application using any programming language** like Java or Python. For example, JDBC (Java Database Connectivity) can connect the Java application to the MySQL database. In contrast, PyMySQL can be used in Python to connect to the MySQL database.
The GUI will be designed to make it easy for users to interact with the database using buttons, text fields, and other user interface components. The students can use any GUI development tools, such as JavaFX or Swing in Java, to create the interface. In Python, the Tkinter library can create a graphical interface to communicate with the MySQL database.
Once the GUI is created, users can register by providing their details, which will be inserted into the MySQL database using the insert statement. Users can then log in using their registered credentials, and the GUI should authenticate them by querying the MySQL database using the select statement. Users should also be able to update their details and delete their accounts using the update and delete statements.
Overall, Task 4 requires you to create a functional GUI that interacts with the MySQL database created in Task 3, enabling users to perform basic CRUD (Create, Read, Update, Delete) operations on the database.
Note: Here are a few more programming language examples and the corresponding database connections that can be used for Task 4:
C# - You can use the .NET Framework's ADO.NET library to connect to a MySQL database. The connection string looks something like this:
```
string connString = "SERVER=localhost;DATABASE=myDatabase;UID=myUsername;PASSWORD=myPassword;";
```

PHP - PHP has built-in support for MySQL databases through the MySQLi and PDO extensions. Here is an example connection string using MySQLi:
```
$conn = mysqli_connect("localhost", "myUsername", "myPassword", "myDatabase");
```

Ruby - You can use the Ruby MySQL library to connect to a MySQL database. Here is an example connection code:
```
require 'mysql2'
client = Mysql2::Client.new(
  host: 'localhost',
  username: 'myUsername',
  password: 'myPassword',
  database: 'myDatabase'
)
```

Node.js - You can use the mysql2 library in Node.js to connect to a MySQL database. Here is an example code:

```
const MySQL = require('mysql2/promise');
const pool = mysql.createPool({
  host: 'localhost',
  user: 'myUsername',
  password: 'myPassword',
  database: 'myDatabase'
});
```

These are just a few examples, and many other programming languages and libraries can be used to connect to a MySQL database.

Task 5: Perform Select, Insert, Update, and Delete Statements Through GUI

In this task, you must ensure that the GUI you created in task 4 can perform the necessary SQL statements for registration and login scenarios. This approach means that application users can interact with the database using the GUI interface to perform select, insert, update, and delete operations.

For example, a user might use the GUI to input their registration details, such as their name, email address, and password. The application will then perform an insert statement on the database to add the user's details to the relevant table. Similarly, users might use the GUI to input their login details, such as their email address and password. The application will then perform a select statement on the database to check if the user's details match those in the relevant table and, if so, allow the user to log in.

You will need to ensure that the necessary SQL statements are written and implemented in the backend code of the application, allowing the GUI to communicate with the database and perform the desired operations. This approach may involve using libraries and frameworks specific to the programming language you have chosen to create the application and implementing error handling and validation to ensure data integrity and prevent security vulnerabilities.

Conclusion:

In conclusion, developing a user-friendly database system that allows XYZ university students to register and manage their information efficiently is a crucial step for University XYZ in today's digital age. By implementing this project, the university is taking a significant step towards improving the overall academic experience for its students. The new system will make it easier for students to register and manage their information and help the university staff streamline administrative processes, reduce errors, and enhance data security. With the successful implementation of this project, the university can expect to see increased student satisfaction, improved administrative efficiency, and a better overall academic environment. This project represents an essential investment in the university's future and will undoubtedly positively impact students and staff.

What should you deliver?

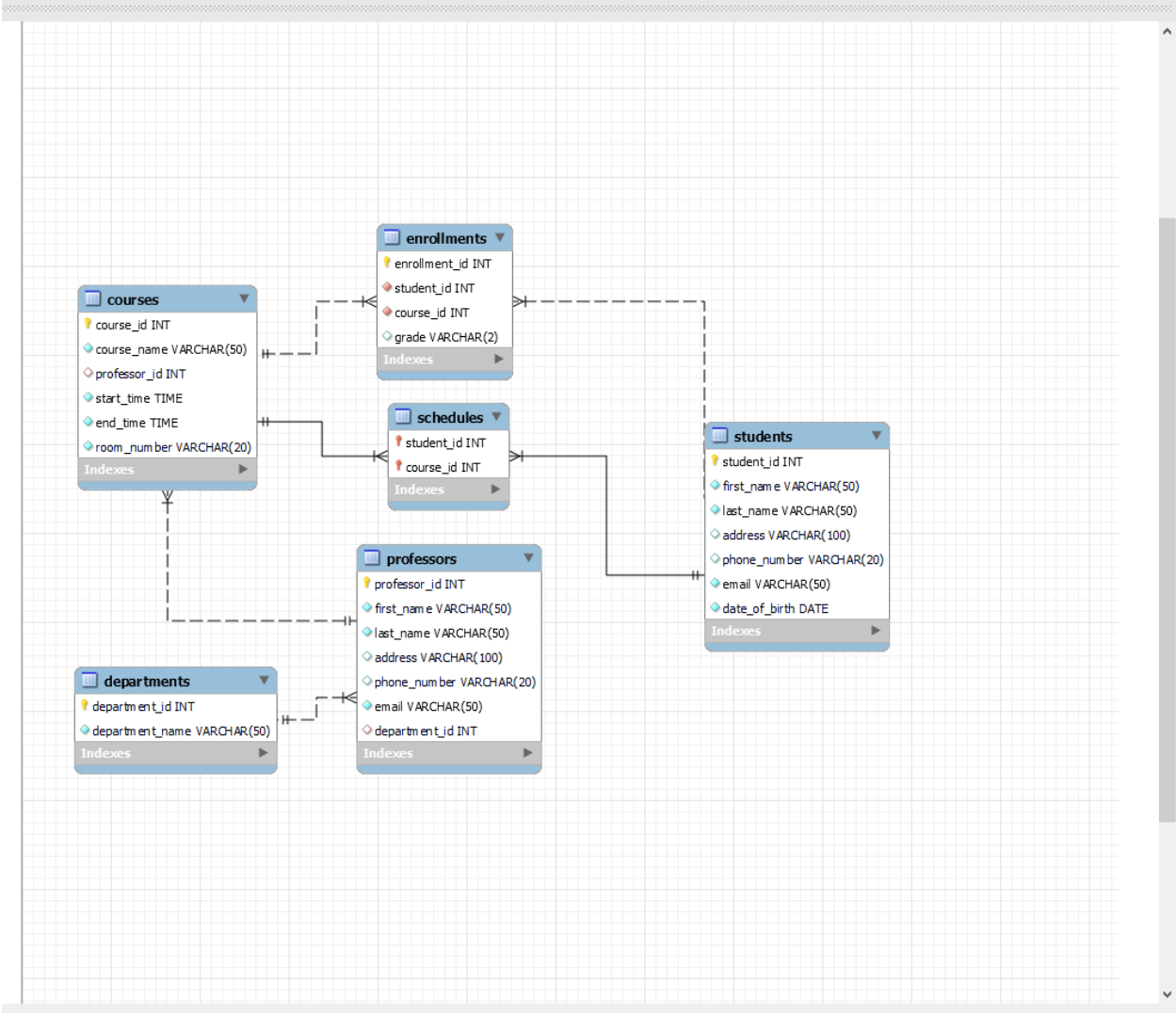
1. Write all the entity names and their columns and other information similar to how I wrote in the **Entities** section .

YOUR ANSWER:

Student - This entity represents a student and includes attributes such as student ID, name, address, phone number, email address, and date of birth.

Course - This entity represents a course and includes attributes such as course ID, course name, instructor name, start time, end time, and room number.
Enrolment - This entity represents a student's enrolment in a course and includes attributes such as enrolment ID, student ID, course ID, and grade.
Schedule - This entity represents a student's schedule and includes attributes such as student ID, and course ID.
Department - This entity represents a department and includes attributes such as department ID and name.
Professor - This entity represents a professor and includes attributes such as professor ID, name, address, phone number, and email address

2. Perform task 1 and capture an image of the Lucidchart's conceptual E-R model for the database and add it to this section.
YOUR ANSWER:



3. Perform task 2 and add all the images and SQL codes related to creating the relational model for the database in MySQL in this section

YOUR ANSWER:

```
CREATE DATABASE Project1;

USE Project1;

CREATE TABLE students (
  student_id INT PRIMARY KEY AUTO_INCREMENT,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  address VARCHAR(100),
  phone_number VARCHAR(20),
  email VARCHAR(50) UNIQUE NOT NULL,
  date_of_birth DATE NOT NULL
);

CREATE TABLE departments(
  department_id INT PRIMARY KEY AUTO_INCREMENT,
  department_name VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE professors(
  professor_id INT PRIMARY KEY AUTO_INCREMENT,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  address VARCHAR(100),
  phone_number VARCHAR(20),
  email VARCHAR(50) UNIQUE NOT NULL,
  department_id INT,
  FOREIGN KEY (department_id) REFERENCES departments(department_id)
);

CREATE TABLE courses(
  course_id INT PRIMARY KEY AUTO_INCREMENT,
  course_name VARCHAR(50) NOT NULL,
  professor_id INT,
  start_time TIME NOT NULL,
  end_time TIME NOT NULL,
  room_number VARCHAR(20) NOT NULL,
  FOREIGN KEY (professor_id) REFERENCES professors(professor_id)
);

CREATE TABLE enrollments(
  enrollment_id INT PRIMARY KEY AUTO_INCREMENT,
  student_id INT NOT NULL,
  course_id INT NOT NULL,
  grade VARCHAR(2),
```

```
  FOREIGN KEY (student_id) REFERENCES students(student_id),
  FOREIGN KEY (course_id) REFERENCES courses(course_id)
);

CREATE TABLE schedules(
  student_id INT NOT NULL,
  course_id INT NOT NULL,
  PRIMARY KEY (student_id, course_id),
  FOREIGN KEY (student_id) REFERENCES students(student_id),
  FOREIGN KEY (course_id) REFERENCES courses(course_id)
);

alter table students
add column password varchar(255) not null;
```

Project1* departments enrollment* professor*

Limit to 1000 rows

```
CREATE DATABASE Project1;

USE Project1;

CREATE TABLE students (
  student_id INT PRIMARY KEY AUTO_INCREMENT,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  address VARCHAR(100),
  phone_number VARCHAR(20),
  email VARCHAR(50) UNIQUE NOT NULL,
  date_of_birth DATE NOT NULL
);

CREATE TABLE departments(
  department_id INT PRIMARY KEY AUTO_INCREMENT,
  department_name VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE professors(
  professor_id INT PRIMARY KEY AUTO_INCREMENT,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  address VARCHAR(100),
  phone_number VARCHAR(20),
  email VARCHAR(50) UNIQUE NOT NULL,
  department_id INT,
  FOREIGN KEY (department_id) REFERENCES departments(department_id)
);
```

```sql
CREATE TABLE courses(
  course_id INT PRIMARY KEY AUTO_INCREMENT,
  course_name VARCHAR(50) NOT NULL,
  professor_id INT,
  start_time TIME NOT NULL,
  end_time TIME NOT NULL,
  room_number VARCHAR(20) NOT NULL,
  FOREIGN KEY (professor_id) REFERENCES professors(professor_id)
);

CREATE TABLE enrollments(
  enrollment_id INT PRIMARY KEY AUTO_INCREMENT,
  student_id INT NOT NULL,
  course_id INT NOT NULL,
  grade VARCHAR(2),
  FOREIGN KEY (student_id) REFERENCES students(student_id),
  FOREIGN KEY (course_id) REFERENCES courses(course_id)
);

CREATE TABLE schedules(
  student_id INT NOT NULL,
  course_id INT NOT NULL,
  PRIMARY KEY (student_id, course_id),
  FOREIGN KEY (student_id) REFERENCES students(student_id),
  FOREIGN KEY (course_id) REFERENCES courses(course_id)
);

alter table students
add column password varchar(255) not null;
```

4. Perform task 3 and write all the SQL queries for SELECT, INSERT, UPDATE, and DELETE statements of the register and login page of the system.

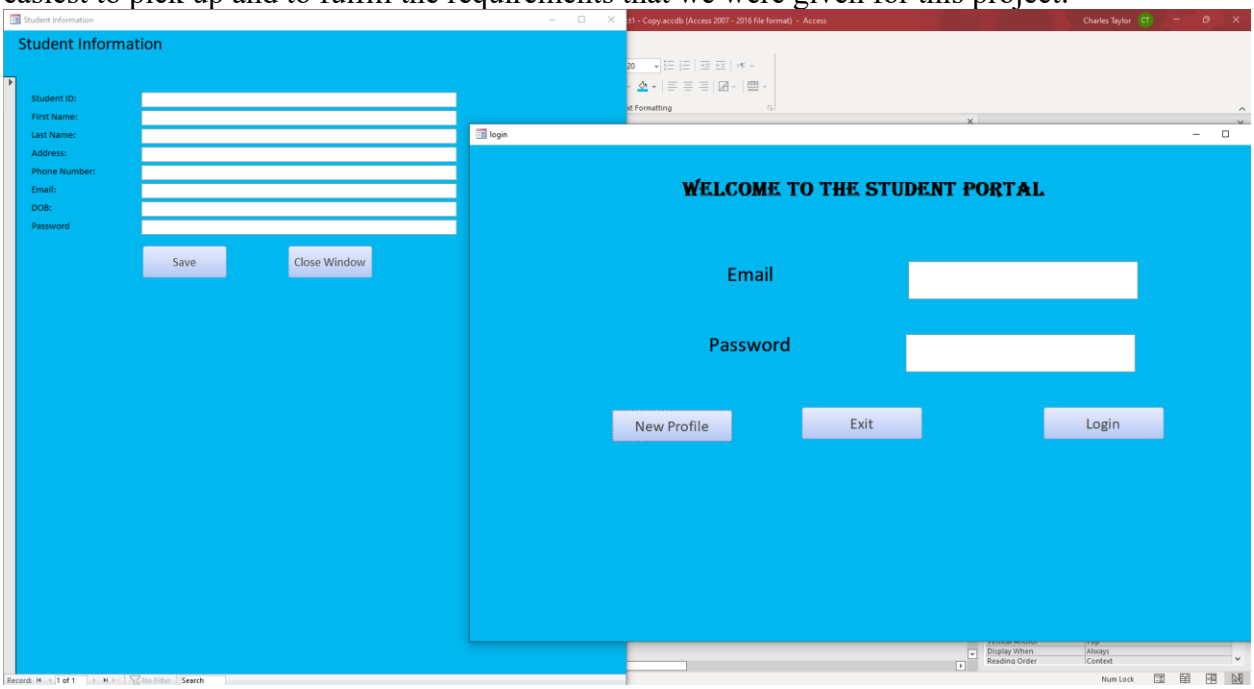
YOUR ANSWER:

- INSERT INTO `project1`.`departments` (`department_name`) VALUES ('Math');
- UPDATE `project1`.`enrollments` SET `grade` = 'B' WHERE (`enrollment_id` = '1');
- UPDATE `project1`.`enrollments` SET `grade` = 'B' WHERE (`enrollment_id` = '2');
- DELETE FROM `project1`.`professors` WHERE (`professor_id` = '1003');
- SELECT courses.course_id, courses.course_name, professors.last_name, courses.start_time, courses.end_time, courses.room_number FROM courses INNER JOIN professors ON courses.professor_id = professors.professor_id;
- SELECT schedules.student_id, schedules.course_id AS schedules_course_id, courses.course_id AS courses_course_id, courses.course_name, courses.professor_id, courses.start_time, courses.end_time, courses.room_number FROM courses INNER JOIN schedules ON courses.[course_id] = schedules.[course_id];

5. Perform task 4 and write what language did you use? What connectivity API did you use? What is your connection string using the chosen programming language? Why did you use this programming language? Capture the screen of the register and login page of the software application you designed .

YOUR ANSWER:

I used My SQL for my database. And for my front end/ GUI I used Microsoft Access that uses Visual Basic for Applications code. For connectivity I use ODBC connection to sync My SQL to Access. The reason why I use Microsoft Access is because given the amount of time that we were allotted for this project and the lack of knowledge of both My SQL and another programming language that can be synced to My SQL. Microsoft Access seemed to be the easiest to pick up and to fulfill the requirements that we were given for this project.



6. Include all the source codes related to performing SELECT, INSERT, UPDATE, and DELETE Statements for the register and login page. The source should show the SQL commands in MySQL and the commands used in your programming language to call these SQL statements.

YOUR ANSWER:

```
INSERT INTO students (first_name, last_name, address, phone_number, email, date_of_birth)
VALUES ('John', 'Doe', '123 Main St', '555-1234', 'john.doe@example.com', 2/21/2023);


SELECT * FROM students;


SELECT * FROM students WHERE email = 'hot@mail.com';


UPDATE students
SET address = '456 New St'
```

```
WHERE student_id = 1;

DELETE FROM students WHERE student_id = 1;
```

Mircosoft Visual Basic for Application

Login in Form

```
Private Sub Command10_Click()

   Dim userLevel As String
   Dim studentID As Variant

   If IsNull(Me.email) Then
      MsgBox "Please enter the correct Student ID.", vbInformation, "Student ID required"
      Me.email.SetFocus

   ElseIf IsNull(Me.password) Then
      MsgBox "Please enter the correct password.", vbInformation, "Password required"
      Me.password.SetFocus

   Else
      If Not (IsNull(DLookup("email", "students", "email='" & Me.email.Value & "")) And _
         Not (IsNull(DLookup("password", "students", "password='" & Me.password.Value &
"")) And _
         DLookup("email", "students", "email='" & Me.email.Value & "") = Me.email.Value And
_
         DLookup("password", "students", "password='" & Me.password.Value & "") =
Me.password.Value Then

         ' Retrieve the Student ID for the current email and password
         studentID = DLookup("student_id", "students", "email='" & Me.email.Value & "' AND
password='" & Me.password.Value & "")

         MsgBox "Access granted... you may proceed!", vbInformation, "Succeeded"

         DoCmd.Minimize

         Dim filterValue As String
         filterValue = Forms!login!email.Value

         ' Open the "student information" form with a filter based on the value from the "logic
form"
         DoCmd.OpenForm "student_main", , , "email = '" & filterValue & "'"

      Else
         MsgBox "Email or Password is incorrect", vbCritical, "Login denied"
         Me.email = ""
```

```
            Me.password = ""
            Me.email.SetFocus

        End If

    End If


End Sub

Private Sub Command11_Click()

    DoCmd.Close

End Sub
```

Schedules Form

```
Option Compare Database
Option Explicit



Private Sub add_button_Click()
    Dim studentID As Long
    Dim courseID As Long

    ' Get the student_id and course_id from the text boxes
    studentID = CLng(Me.student_id.Value)
    courseID = CLng(Me.course_id.Value)

    ' Create a SQL query to add a new record with the specified student_id and course_id
    Dim addQuery As String
    addQuery = "INSERT INTO schedules (student_id, course_id) VALUES (" & studentID & ",
" & courseID & ");"

    ' Execute the SQL query
    CurrentDb.Execute addQuery, dbFailOnError

    ' Show a confirmation message
    MsgBox "Class added is successfully.", vbInformation, "Success"

End Sub

Private Sub delete_button_Click()
    Dim studentID As Long
```

```
    Dim courseID As Long

    ' Get the student_id and course_id from the text boxes
    studentID = CLng(Me.student_id.Value)
    courseID = CLng(Me.course_id.Value)

    ' Create a SQL query to delete the record with the specified student_id and course_id
    Dim deleteQuery As String
    deleteQuery = "DELETE FROM schedules WHERE student_id = " & studentID & " AND
course_id = " & courseID & ";"

    ' Execute the SQL query
    CurrentDb.Execute deleteQuery, dbFailOnError

    ' Show a confirmation message
    MsgBox "Class deleted is successfully.", vbInformation, "Success"
End Sub

Private Sub Form_Load()
    'test"

    ' Set the value of txtStudentID to the student_id value
    Me.student_id.Value = Forms!student_main!student_id.Value

End Sub
```

Student_main Form

```
Option Compare Database

Option Explicit



Private Sub Command14_Click()
    DoCmd.OpenForm "schedules"

End Sub

Private Sub Command17_Click()

    DoCmd.OpenForm "Course_report"
End Sub

Private Sub Command4_Click() 'dont touch"
```

```
    ' Get the filter value from the "logic form"
    Dim filterValue As String
    filterValue = Forms!login!email.Value

    ' Open the "student information" form with a filter based on the value from the "logic form"
    DoCmd.OpenForm "student_information", , , "email = '" & filterValue & "'"
End Sub



Private Sub Text24_Enter()
    Dim filterValue As String
    filterValue = Forms!login!email.Value
End Sub

Private Sub my_schedules_button_Click()

    DoCmd.OpenForm "schedules Query"

End Sub
```

8. Write any obstacles, issues, problems, difficulties, etc., you encountered during the design, development, implementation, and demonstration of the Student Registrations System at XYZ University

When it came to MySQL it was fairly easy to maneuver around once one I had learned the basics. But when I got to use Microsoft Access, the difficulty got ramped up. I had to spend a couple of days watching YouTube videos of people maneuver around access and learning how to link MySQL to access. Once when I got my bearings off how to make forms and manipulate my queries the biggest hurdle that I face was my lack of knowledge of using visual basic for applications (VBA) code and by the time I realized I was over my head I was already too deep to change to another programming language. I was able to perform majority of the tests that was requested in this project. But I ran into some issues with producing a forum that sink my student login page student ID to that form so I can query the students the schedule to that particular student but beyond that I'm pretty proud of what I was able to accomplish given the amount of time and knowledge.